

Tema 4.

Análisis automático de textos para revisión lingüística

Informe automático de divergencias en textos paralelos monolingües

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

Contexto profesional.....	1
Qué vas a aprender en esta unidad.....	2
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	3
1.1. Textos paralelos monolingües.....	3
1.2. Por qué automatizar parte de la revisión.....	3
1.3. Del corpus al informe: una cadena de análisis.....	4
1.4. Revisión lingüística, calidad y evidencia.....	6
1.5. Corpus, texto y unidad de análisis.....	7
1.6. Segmentación en oraciones.....	7
1.7. Frecuencia léxica y coherencia terminológica.....	8
1.8. Patrones formales: espacios, puntuación y normalización.....	8
1.9. Expresiones regulares como reglas transparentes.....	9
1.10. Posibles calcos y estructuras con a + infinitivo.....	10
1.11. Gerundios como candidatos a revisión.....	10
1.12. Falsos positivos, falsos negativos y criterio lingüístico.....	11
1.13. Funciones, informes y trazabilidad.....	12
1.14. Límites de los métodos basados en reglas.....	12
2. Análisis automático de textos para revisión lingüística con Python.....	12
2.1. Leer textos como datos.....	12
2.2. Rutas de archivos y pathlib.....	13
2.3. Cadenas, listas y corpus pequeño.....	14
2.4. Segmentar en oraciones con split y strip.....	15
2.5. Contar palabras y normalizar con minúsculas.....	15
2.6. Recorrer varias versiones con enumerate.....	16
2.7. Detectar ausencia de espacio después de puntuación.....	17
2.8. Recuperar contexto con índices y slicing.....	18
2.9. Localizar dobles espacios.....	18
2.10. Comparación terminológica básica.....	19
2.11. Tablas de resultados.....	22
2.12. Comparar términos en muchas versiones.....	22
2.13. Posibles galicismos: buscar candidatos a + infinitivo.....	23
2.14. Buscar gerundios y contexto inmediato.....	25
2.15. Organizar el análisis con funciones.....	27
2.16. Devolver resultados con return.....	30
2.17. Generar un informe final.....	32
2.18. Cómo leer los resultados antes de corregir.....	33
Reglas simples frente a análisis lingüístico avanzado.....	33
Buenas prácticas para revisar textos con Python.....	34
Errores frecuentes de aprendizaje.....	34

Síntesis final.....	35
Cierre narrativo.....	35
Referencias.....	35

Título narrativo

Revisar dos versiones de un mismo texto exige hoy algo más que leer con atención: hay que saber convertir el texto en datos, detectar indicios y devolver al equipo humano una lista razonada de lugares que merecen revisión. En esta unidad, la revisión lingüística se plantea como una tarea profesional híbrida, en la que el criterio sigue siendo humano y Python ayuda a localizar con rapidez patrones repetitivos, incoherencias terminológicas y señales formales que pueden pasar inadvertidas cuando el volumen de texto crece.

Contexto profesional

Imagina que te incorporas al equipo lingüístico de **Verba Pública**, una institución —podría tratarse de un organismo multilateral, de un organismo público o de una gran empresa— que trabaja con documentación institucional en varias lenguas. La institución ha encargado la traducción de un documento oficial a varios proveedores, y las versiones llegan en el mismo idioma, pero no son idénticas: una habla de *equipo* donde otra habla de *grupo*; una prefiere *compañía* y otra *empresa*; una mantiene una puntuación impecable y otra acumula pequeños problemas de espaciado.

La pregunta profesional va más allá de cuál de las versiones «suena mejor», porque el equipo necesita tomar decisiones justificables:

- Qué versión puede validarse como base.
- Qué términos deben incorporarse a un glosario consolidado.
- Qué proveedor respeta mejor la política terminológica de la institución.
- Qué problemas formales conviene corregir antes de publicar el documento.
- Qué patrones lingüísticos necesitan revisión humana especializada.

Más que una lectura subjetiva aislada, la institución quiere un procedimiento repetible que ayude a responder preguntas concretas:

- ¿Hay divergencias terminológicas entre versiones?
- ¿Aparecen patrones formales que sugieran falta de revisión, como espacios dobles o ausencia de espacio tras puntuación?
- ¿Se repiten estructuras que podrían requerir criterio lingüístico, como ciertos usos de *a* + infinitivo o gerundios?
- ¿Puede generarse un pequeño informe que reúna los hallazgos sin tener que revisar manualmente todo el corpus desde cero?

En una revisión manual tradicional, una lingüista tendría que leer las versiones una al lado de la otra, oración por oración, y marcar las diferencias. Esa lectura sigue siendo imprescindible, porque interpretar un texto exige criterio lingüístico, pero cuando el volumen crece, la revisión exclusivamente manual se vuelve lenta, costosa y difícil de documentar.

Python permite construir una primera capa de revisión automática que, sin reemplazar la lectura profesional, ayuda a organizarla: localiza indicios, cuenta fenómenos, compara usos y genera un informe preliminar que orienta el trabajo humano. De ahí una idea de fondo de este tema que vale para todo el PLN: un texto puede leerse como discurso, pero también puede representarse como dato.

Este manual desarrolla esa mirada: cómo preparar un pequeño sistema de análisis textual para revisar versiones paralelas monolingües, detectar divergencias terminológicas y localizar posibles problemas de estilo, puntuación y gramática.

Qué vas a aprender en esta unidad

Al terminar esta unidad deberías poder explicar, antes de ejecutar código, por qué cada operación es relevante para una tarea de revisión lingüística. En concreto, aprenderás a:

- entender qué son los textos paralelos monolingües en el contexto de una revisión comparativa;
- distinguir entre indicio automático, error lingüístico y decisión profesional;
- leer archivos de texto en Python controlando la codificación;
- representar un corpus pequeño como una lista de textos;
- segmentar textos en oraciones con procedimientos simples y reconocer sus limitaciones;
- normalizar texto con operaciones como minúsculas para facilitar comparaciones;
- contar palabras o expresiones como primera aproximación a la coherencia terminológica;
- recorrer muchos textos con bucles y `enumerate`;
- usar expresiones regulares para localizar patrones formales;
- recuperar contexto alrededor de una coincidencia;
- comparar versiones paralelas oración a oración;
- organizar comprobaciones en funciones reutilizables;
- diferenciar entre mostrar un resultado con `print` y devolverlo con `return`;
- reunir varios análisis parciales en un informe final interpretable.

Mapa conceptual de la unidad

Tabla 1. Mapa conceptual de la unidad: bloques, preguntas lingüísticas y herramientas

Bloque	Pregunta lingüística	Herramienta de Python	Resultado esperado
Textos paralelos monolingües	¿Cómo comparo dos versiones españolas de un mismo contenido?	Listas, índices, <code>zip</code>	Correspondencia entre segmentos
Lectura de archivos	¿Cómo convierto documentos externos en texto procesable?	<code>open</code> , <code>encoding="utf-8"</code> , <code>pathlib.Path</code>	Cadenas de texto cargadas en memoria
Segmentación	¿Dónde empieza y termina cada unidad revisable?	<code>split</code> , <code>strip</code> , listas por comprensión	Lista de oraciones o segmentos
Frecuencia léxica	¿Qué términos se repiten y en qué versiones?	<code>lower</code> , <code>count</code> , bucles	Recuentos comparables
Patrones formales	¿Dónde hay señales mecánicas de posible error?	<code>in</code> , <code>finditer</code> , índices, <i>slicing</i>	Posiciones y contexto
Expresiones regulares	¿Cómo busco estructuras textuales flexibles?	Módulo <code>re</code> , patrones, grupos	Coincidencias interpretables
Funciones	¿Cómo evito repetir análisis manualmente?	<code>def</code> , parámetros, <code>return</code>	Comprobaciones reutilizables
Informe	¿Cómo convierto hallazgos dispersos en evidencia?	Funciones combinadas, tablas, texto formateado	Informe de revisión preliminar

1. Marco teórico

1.1. Textos paralelos monolingües

En corpus lingüísticos y estudios de traducción, un corpus paralelo suele entenderse como un conjunto de textos alineados en dos o más lenguas, normalmente un original y su traducción, mientras que un corpus comparable reúne textos que comparten rasgos relevantes, como el dominio, el género o el periodo, sin ser necesariamente traducciones directas entre sí (Olohan, 2004; Zanettin, 2012). La unidad trabaja con una adaptación pedagógica de esa lógica: los **textos paralelos monolingües**, es decir, versiones en la misma lengua que representan un mismo contenido o una misma función comunicativa y que pueden compararse por secciones, párrafos u oraciones.

La expresión «paralelo monolingüe» debe entenderse aquí con cuidado. No designa una categoría universal cerrada en todas las tradiciones de corpus; la unidad aprovecha la idea de paralelismo textual para una tarea profesional concreta, la de comparar versiones españolas de un mismo documento. En revisión institucional esto es habitual: puede haber dos proveedores que traducen o adaptan un texto, dos versiones sucesivas de un informe, una versión revisada y otra sin revisar, o variantes regionales de un mismo contenido.

El interés lingüístico de estos textos reside en la divergencia. Si dos versiones dicen exactamente lo mismo con las mismas palabras, la comparación automática aporta poco; si difieren, esas diferencias pueden revelar decisiones de estilo, terminología, registro, puntuación o estructura sintáctica. Tampoco hay que asumir que toda diferencia es mala, porque la variación es normal y, en muchos casos, legítima: lo importante es distinguir entre variación aceptable, incoherencia terminológica y posible error.

Tabla 2. Tipos de divergencia entre versiones paralelas monolingües

Tipo de divergencia	Ejemplo	Interpretación posible
Terminológica	<i>equipo</i> frente a <i>grupo</i>	Puede ser variación aceptable o falta de glosario común
Ortográfica o mecánica	«empresa,el»	Probable problema formal por ausencia de espacio
Estilística	<i>llevar a cabo</i> frente a <i>realizar</i>	Puede responder a preferencias de estilo
Sintáctica	«decidió a revisar»	Puede ser candidato a revisión según contexto
Discursiva	Una versión divide en dos oraciones y otra en una	Puede afectar claridad o ritmo

Para una lingüista computacional en formación, conviene retener que el texto se convierte en un objeto analizable: una versión puede representarse como una cadena de caracteres; varias versiones pueden guardarse en una lista; cada oración puede ocupar una posición, cada término puede contarse y cada patrón puede buscarse. Esa representación no agota el análisis lingüístico, pero lo hace operativo.

1.2. Por qué automatizar parte de la revisión

La revisión lingüística y la revisión de traducción son tareas expertas que requieren comprensión, competencia textual, conocimiento del dominio y criterio profesional. Las normas de servicios de traducción insisten en procesos de revisión y verificación que van mucho más allá de una comprobación mecánica (International Organization for Standardization [ISO], 2015), y la evaluación de la calidad de una traducción exige igualmente criterios definidos y categorías de error o de adecuación, más que impresiones generales (ISO, 2024; Lommel *et al.*, 2014).

Automatizar parte de la revisión no supone automatizar la decisión final: supone delegar en el programa las operaciones repetitivas, sistemáticas y fácilmente verificables. Python puede contar cuántas veces aparece un término, localizar dobles espacios, detectar una coma sin espacio posterior o listar estructuras candidatas a revisión, mientras que la persona especialista conserva la responsabilidad de interpretar esos hallazgos.

Este reparto de tareas tiene varias ventajas. En primer lugar, reduce la carga de búsqueda manual: en un texto largo, encontrar todas las ocurrencias de una palabra o todos los espacios dobles es una tarea tediosa y propensa a omisiones. En segundo lugar, permite reconstruir el procedimiento, porque si se usa el mismo patrón de búsqueda puede repetirse en otra versión o en otro corpus. En tercer lugar, facilita la documentación de decisiones, ya que un informe con recuentos, ejemplos y posiciones permite discutir con más precisión.

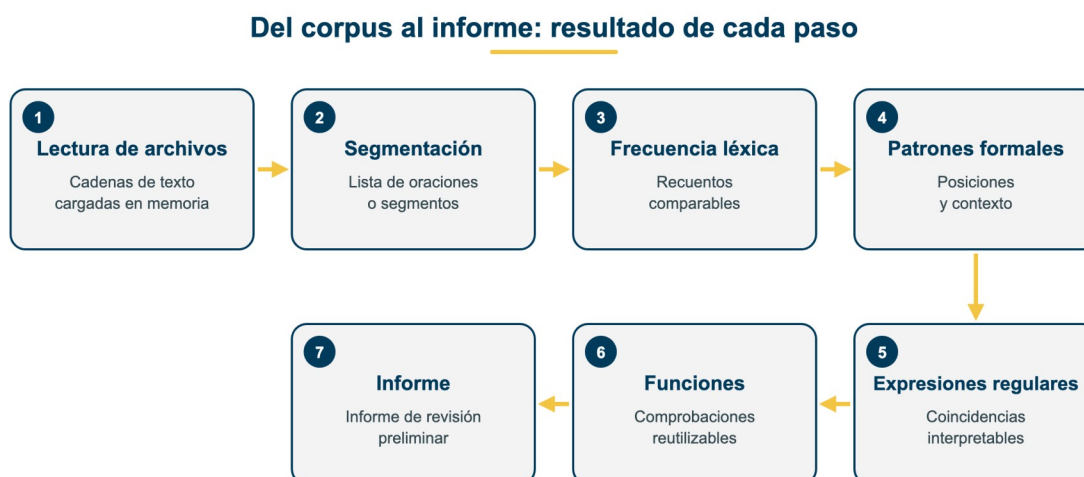
También existen riesgos. Un detector basado en reglas puede producir falsos positivos, es decir, casos señalados por el programa que no son errores reales, y falsos negativos, casos problemáticos que no se detectan porque no cumplen el patrón previsto. Por eso conviene hablar de **candidatos a revisión** y no de errores automáticos: el lenguaje es demasiado variable para que una regla simple capture todos los matices.

En Verba Pública, esta idea se traduce en una regla profesional clara: el informe automático se entrega como mapa de lectura, nunca como veredicto. La revisora sénior recibe una lista de «lugares que conviene mirar» en lugar de una lista de «fallos», una diferencia que protege la calidad del trabajo lingüístico y evita que la herramienta aparente más certeza de la que realmente tiene.

1.3. Del corpus al informe: una cadena de análisis

Una tarea de PLN empieza mucho antes del informe final, con una cadena de decisiones que en esta unidad puede resumirse así: recopilar textos, leerlos correctamente, normalizarlos cuando sea necesario, segmentarlos, aplicar comprobaciones, guardar resultados, interpretarlos y comunicarlos. Cada eslabón condiciona los siguientes.

Figura 1. Cadena de análisis para la revisión de versiones paralelas monolingües



Nota. Elaboración propia a partir del mapa conceptual de la unidad.

Tabla 3. Cadena de análisis desde el corpus hasta el informe

Fase	Pregunta	Decisión técnica	Riesgo si se descuida
Entrada	¿De dónde viene el texto?	Leer archivos .txt indicando la codificación	Caracteres dañados o errores de lectura
Representación	¿Cómo guardo una o varias versiones?	Cadenas, listas, diccionarios si procede	Datos desordenados o difíciles de comparar
Normalización	¿Qué diferencias quiero ignorar?	Pasar a minúsculas, limpiar espacios	Comparaciones injustas o pérdida de información
Segmentación	¿Qué unidad reviso?	Oraciones, párrafos, líneas	Coincidencias sin contexto
Análisis	¿Qué busco?	Recuentos, patrones, regex, funciones	Resultados irrelevantes
Interpretación	¿Qué significa el hallazgo?	Criterio lingüístico	Confundir señal con error
Informe	¿Cómo lo comunico?	Tablas, ejemplos, resumen	Evidencia difícil de usar

La literatura de corpus insiste en que el corpus es un recurso construido para responder preguntas concretas, y no una colección neutra de textos (McEnery y Hardie, 2012). En una revisión profesional, esa idea pesa mucho: las versiones se recopilan para verificar la coherencia, detectar divergencias y justificar decisiones.

El informe final debe preservar la relación entre dato y contexto. Un recuento aislado puede ser útil, pero no basta: saber que *implementación* aparece cinco veces en una versión y una vez en otra puede sugerir una divergencia terminológica, pero la revisión necesita mirar dónde aparece, qué función cumple y si existe una alternativa aceptada por el glosario. Por eso el análisis automático debe tender a producir resultados contextualizados.

Por ejemplo, si una herramienta detecta diez casos de *a* + infinitivo, no puede afirmar automáticamente que todos sean galicismos: algunos serán correctos y otros dependerán del sustantivo anterior, del régimen preposicional o de la construcción completa. La herramienta reduce el espacio de búsqueda; la lingüista interpreta.

Esta relación entre automatización y revisión humana es esencial en los entornos profesionales:

Tabla 4. Tareas automáticas y decisiones lingüísticas posteriores

Tarea automática	Decisión lingüística posterior
Contar ocurrencias de <i>empresa</i> y <i>compañía</i>	Decidir cuál debe incorporarse al glosario
Detectar un punto o una coma seguidos de una letra sin espacio	Confirmar si hay error tipográfico real
Localizar gerundios	Clasificar usos correctos, de posterioridad o especificativos
Comparar términos por versión	Valorar la coherencia, el estilo y la adecuación al cliente
Generar un informe preliminar	Priorizar la revisión humana

Para una lingüista de segundo curso, esta unidad presenta una idea muy poderosa: programar para PLN consiste en **formalizar una pregunta lingüística**, y no solo en «hacer que algo funcione».

1.4. Revisión lingüística, calidad y evidencia

La calidad textual no es una propiedad única. En traducción y comunicación especializada suele relacionarse con adecuación al encargo, fidelidad funcional, coherencia terminológica, corrección lingüística, legibilidad y cumplimiento de convenciones del cliente o institución. Las normas profesionales, como ISO 17100 para servicios de traducción, subrayan la importancia de procesos definidos, competencias profesionales y revisión por una persona distinta cuando procede (ISO, 2015).

La evaluación de la calidad también puede apoyarse en marcos de categorías. Por ejemplo, MQM propone describir los problemas de calidad mediante dimensiones y tipos de error, lo que permite pasar de una impresión global a una anotación más sistemática (Lommel *et al.*, 2014). ISO 5060 actualiza este terreno con una norma para evaluar resultados de traducción, atenta a los criterios, las especificaciones y la finalidad de la evaluación (ISO, 2024).

La unidad no pretende que el alumnado aplique una taxonomía completa de calidad; busca que entienda una idea fundamental: un buen análisis automático debe estar al servicio de una pregunta de revisión. En lugar de buscar «todo lo raro», se buscan indicios relacionados con una necesidad profesional, que en el caso de Verba Pública consiste en comparar versiones españolas para detectar divergencias antes de consolidar una versión final.

Esta perspectiva evita dos errores frecuentes. El primero es tecnicista: creer que si Python detecta algo, ese algo ya es un error. El segundo es antitécnico: pensar que, como el criterio final es humano, la automatización no aporta valor. Entre ambos extremos se encuentra la práctica profesional madura, que usa las herramientas para observar mejor y decidir con más fundamento.

1.5. Corpus, texto y unidad de análisis

Un corpus es un conjunto de textos reunidos con un propósito. Puede ser grande o pequeño, especializado o general, monolingüe o multilingüe, equilibrado o diseñado para una tarea muy concreta. Lo importante es que permite estudiar patrones de uso a partir de datos. En PLN, un corpus pequeño puede ser suficiente para aprender procedimientos; en investigación o producción profesional, el tamaño y diseño del corpus deben justificarse con más cuidado (McEnery y Hardie, 2012; Olohan, 2004).

En esta unidad, el corpus es pequeño y controlado: varias versiones de textos de trabajo. Esa escala es didáctica, pero no trivial, porque muchas tareas profesionales reales comienzan con corpus modestos: un contrato, un informe, una guía de estilo, una batería de páginas web o un conjunto de documentos institucionales. En esos casos, Python sirve para apoyar una revisión concreta, más que para descubrir leyes generales de la lengua.

La unidad de análisis puede cambiar. A veces interesa el texto completo, por ejemplo para contar cuántas veces aparece un término. Otras veces interesa la oración, porque permite localizar un problema en contexto. En otros proyectos podrían interesar párrafos, segmentos alineados, títulos, notas al pie o unidades terminológicas. Elegir la unidad de análisis es una decisión lingüística antes que técnica.

Tabla 5. Unidades de análisis: ventajas y limitaciones

Unidad	Ventaja	Limitación
Texto completo	Permite recuentos globales	Puede ocultar dónde ocurre el fenómeno
Párrafo	Mantiene contexto discursivo	Puede ser demasiado amplio para revisión puntual
Oración	Facilita localización y comparación	La segmentación automática puede fallar
Palabra	Permite frecuencias y concordancias	No capta siempre expresiones complejas
Patrón	Localiza estructuras específicas	Depende de una formulación precisa

1.6. Segmentación en oraciones

Segmentar un texto consiste en dividirlo en unidades menores. En la unidad se trabaja con una segmentación muy simple basada en el punto, `texto.split(".")`, comprensible para empezar pero con limitaciones evidentes: no todos los puntos cierran oraciones, pues pueden aparecer en abreviaturas, números, siglas, iniciales o direcciones, y una oración puede terminar también con signos de interrogación, exclamación o puntos suspensivos.

La segmentación de oraciones es una tarea clásica del procesamiento del lenguaje natural. Kiss y Strunk (2006) mostraron que incluso la detección de límites oracionales puede tratarse con métodos no supervisados y multilingües, precisamente porque no es tan simple como buscar puntos, y los manuales generales de PLN la presentan como parte de la normalización y el preprocesamiento textual (Bird *et al.*, 2009; Jurafsky y Martin, 2026).

En una unidad inicial de programación para lingüistas, usar `split(".")` tiene valor didáctico: permite ver que una cadena puede convertirse en una lista de segmentos y que cada segmento puede limpiarse con

strip. Lo importante es acompañar el procedimiento de una advertencia, la de que estamos ante una aproximación lejos todavía de un segmentador fiable; cuando el proyecto sea real y de mayor escala, convendrá usar herramientas especializadas o reglas más cuidadosas.

La segmentación también afecta a la interpretación. Si el programa localiza un doble espacio dentro de la oración 4, la revisora puede ir directamente a esa zona; si solo informa de que hay tres dobles espacios en todo el texto, el dato es menos útil. Por eso segmentar es algo más que dividir: es hacer que los resultados sean revisables.

1.7. Frecuencia léxica y coherencia terminológica

La frecuencia léxica mide cuántas veces aparece una palabra o expresión. En corpus lingüísticos, las frecuencias permiten observar patrones de uso, comparar variedades, estudiar colocaciones o identificar rasgos de un dominio. En revisión lingüística, los recuentos pueden funcionar como indicios de coherencia o divergencia terminológica.

Si una versión usa *equipo* diez veces y otra usa *grupo* diez veces para referirse a la misma entidad, puede haber una diferencia de criterio; si dentro de una misma versión alternan *compañía*, *empresa* y *sociedad* sin justificación, puede haber un problema de coherencia terminológica. Aun así, el recuento no decide por sí solo, porque puede haber sinónimos legítimos, diferencias de contexto o términos que no sean equivalentes.

La normalización a minúsculas con `lower` facilita las comparaciones básicas, porque permite contar *Empresa* y *empresa* como la misma forma. Pero también supone una decisión, la de ignorar la diferencia entre mayúscula y minúscula, útil en muchos recuentos y problemática en otros, por ejemplo si se comparan nombres propios, siglas o usos institucionales.

Tabla 6. Decisiones de normalización en el recuento léxico

Decisión	Ejemplo	Efecto
Contar con mayúsculas	<code>texto.count("Empresa")</code>	Distingue entre mayúscula y minúscula
Contar en minúsculas	<code>texto.lower().count("empres")</code>	Agrupar variantes de capitalización
Contar una palabra aislada	<i>empresa</i>	Puede incluir parte de formas si no se controla el contexto
Contar expresión	<i>equipo técnico</i>	Más preciso para unidades pluriverbales

En proyectos profesionales, la coherencia terminológica suele relacionarse con glosarios, memorias de traducción, bases terminológicas y guías de estilo (Bowker, 2002). Python no sustituye esos recursos, pero puede ayudar a comprobar si una versión respeta una lista de términos o si dos versiones toman decisiones divergentes.

1.8. Patrones formales: espacios, puntuación y normalización

Los patrones formales son señales de superficie: espacios dobles, puntuación pegada, saltos de línea extraños, caracteres invisibles o incoherencias de formato. A menudo no requieren un análisis lingüístico profundo para detectarse, pero afectan a la calidad percibida del texto y a su preparación para otros procesos.

Un ejemplo claro es la ausencia de espacio después de una coma o un punto: «empresa,el» o «temporales.Utilizando». En español escrito, lo normal es que después de una coma o un punto haya un

espacio si continúa otra palabra, salvo en contextos específicos como números, direcciones web, abreviaturas o ciertos formatos técnicos, de modo que la detección automática debe tener en cuenta que una regla demasiado simple puede señalar casos legítimos.

Los espacios dobles son otro ejemplo: en muchas versiones finales de documentos, un doble espacio entre palabras es una señal mecánica de falta de limpieza, aunque en algunos contextos preformateados, tablas o código puede haber espacios intencionales. De nuevo, la regla detecta una señal y la revisión decide.

La normalización textual puede incluir pasar a minúsculas, eliminar espacios sobrantes, sustituir saltos de línea o unificar comillas, pero normalizar demasiado puede borrar información relevante. En revisión lingüística conviene distinguir entre normalizar para analizar y corregir el texto original: una cosa es crear una copia en minúsculas para contar, y otra muy distinta, modificar el documento final.

1.9. Expresiones regulares como reglas transparentes

Las expresiones regulares son patrones formales que permiten buscar secuencias de caracteres con cierta flexibilidad. El módulo `re` de Python implementa operaciones de búsqueda, coincidencia, sustitución y extracción basadas en estos patrones (Python Software Foundation, 2026c). En PLN, las expresiones regulares se usan para tareas como localizar fechas, abreviaturas, formatos, terminaciones, espacios anómalos o construcciones textuales relativamente previsibles (Bird *et al.*, 2009; Friedl, 2006).

Una expresión regular no «comprende» el texto; se limita a comprobar si una secuencia encaja con una forma. Por ejemplo, el patrón `[. , ] [^\s]` busca un punto o una coma seguidos de un carácter que no sea espacio, lo que puede localizar casos como «texto.Sin» o «empresa,el». El patrón es transparente, porque podemos explicar qué significa cada parte, y esa transparencia es valiosa en contextos docentes y profesionales: la regla puede discutirse, corregirse y documentarse.

Tabla 7. Elementos básicos de las expresiones regulares

Elemento	Significado aproximado	Ejemplo
<code>[. ,]</code>	Un punto o una coma	<code>. o ,</code>
<code>\s</code>	Un espacio o carácter de espacio en blanco	espacio, tabulación, salto de línea
<code>[^\s]</code>	Un carácter que no es espacio	<code>a, E, 5</code>
<code>\w+</code>	Una o más letras, números o guiones bajos	<i>revisar, texto1</i>
<code>\b</code>	Límite de palabra	Final de <i>revisar</i>
<code>`(ar er ir)`</code>	Una de varias terminaciones	<code>ar, er, ir</code>

La ventaja de las expresiones regulares es su precisión formal, y su limitación, que esa precisión no equivale a interpretación lingüística. Un patrón puede encontrar todas las formas acabadas en *-ando* o *-iendo*, pero no puede decidir automáticamente si el gerundio está bien usado en cada contexto. Por eso la unidad las presenta como herramientas para generar candidatos a revisión, nunca como jueces gramaticales.

1.10. Posibles calcos y estructuras con *a* + infinitivo

En español, ciertas secuencias de sustantivo seguido de *a* + infinitivo, como «temas a tratar» o «problemas a resolver», han sido tradicionalmente objeto de comentario en obras normativas y de estilo. La valoración depende del contexto, de la estructura exacta y del grado de asentamiento de la expresión: las obras académicas panhispánicas recomiendan evitar los usos innecesarios cuando exista una alternativa más natural, pero no toda aparición de *a* + infinitivo es incorrecta (Real Academia Española y Asociación de Academias de la Lengua Española, 2005/2023).

La unidad trabaja con un patrón más amplio, que abarca formas como «decidió a revisar» o «intentó a implementar», anómalas en el español estándar según el verbo y el régimen exigido. Desde el punto de vista computacional, lo relevante es que muchas de estas secuencias comparten una forma superficial —una palabra, la preposición *a* y un infinitivo—, y ese parecido formal permite buscarlas con una expresión regular.

Ahora bien, el patrón no debe confundirse con una norma. El español tiene muchas construcciones legítimas con *a* + infinitivo («empezó a revisar», «volvió a leer», «ayudó a preparar», «salió a comprar»), de modo que un detector automático que busque cualquier palabra seguida de *a* y un infinitivo encontrará tanto casos aceptables como dudosos. Por eso el resultado debe llamarse candidato.

Esta distinción es fundamental para estudiantes de lingüística. El programa no sabe de régimen verbal, estructura argumental ni aceptabilidad contextual: solo observa la forma, y es la persona lingüista quien interpreta si esa forma merece revisión a partir de la gramática, el uso, el estilo y la finalidad comunicativa.

1.11. Gerundios como candidatos a revisión

El gerundio en español es una forma no personal del verbo que puede expresar simultaneidad, modo, causa u otras relaciones, según el contexto. La gramática académica describe usos aceptables y usos problemáticos, entre ellos ciertos gerundios de posterioridad o usos adjetivales no asentados (Real Academia Española y Asociación de Academias de la Lengua Española, 2009). En revisión lingüística, los gerundios suelen revisarse porque pueden generar ambigüedad, encadenamiento excesivo o interferencias de estilo.

Detectar gerundios automáticamente parece fácil, porque muchas formas terminan en *-ando*, *-iendo* o *-yendo*, pero una búsqueda por terminación no basta para valorar el uso. «El equipo revisó el informe, incorporando las observaciones» puede requerir lectura contextual; «entró sonriendo» no plantea el mismo tipo de problema, y «leyendo el documento, encontró una incoherencia» funciona de otra manera. La forma no determina por completo la función.

La unidad usa expresiones regulares para localizar posibles gerundios y recuperar las palabras cercanas, porque el contexto inmediato hace más útil la revisión: no es lo mismo devolver solo *utilizando* que mostrar «temporales utilizando una herramienta». Cuanto mejor se preserve el contexto, más fácil será decidir si el candidato exige corrección.

El planteamiento responde a una idea general del PLN basado en reglas: muchas tareas empiezan con patrones simples y mejoran cuando incorporan más información, y el alumnado debe aprender a valorar ese equilibrio. Una regla muy amplia encuentra muchos casos, pero obliga a revisar más falsos positivos; una regla muy estrecha genera menos ruido, pero puede dejar fuera casos relevantes.

1.12. Falsos positivos, falsos negativos y criterio lingüístico

Un falso positivo aparece cuando el sistema señala un caso que no es realmente problemático, y un falso negativo, cuando no señala un caso que sí habría convenido revisar. Ambos conceptos proceden de la evaluación de sistemas de clasificación, pero sirven para pensar cualquier detector basado en reglas: si el patrón es muy general, aumentan los falsos positivos; si es demasiado restrictivo, aumentan los falsos negativos.

Figura 2. Falsos positivos, falsos negativos y alcance de las reglas de detección



En revisión lingüística, los falsos positivos no suponen necesariamente un fracaso. Si el volumen es pequeño y el objetivo es no dejar escapar posibles problemas, puede aceptarse una lista más amplia; en cambio, si el volumen es grande y el equipo dispone de poco tiempo, conviene diseñar patrones más precisos. Por tanto, la calidad de la herramienta depende de la finalidad del encargo.

Tabla 8. Ejemplos de falsos positivos, falsos negativos y reglas de distinto alcance

Situación	Ejemplo	Consecuencia
Falso positivo	El sistema señala «empezó a revisar» como sospechoso	La persona revisora descarta el caso
Falso negativo	El sistema no detecta una incoherencia terminológica expresada con sinónimo	El problema queda fuera del informe
Regla amplia	Busca cualquier a + infinitivo	Mayor cobertura, más ruido
Regla estrecha	Busca solo una lista de verbos problemáticos	Menos ruido, menor cobertura

La competencia lingüística consiste también en saber diseñar preguntas razonables para la máquina. En lugar de la orden vaga de «encuentra errores», Python debe recibir tareas observables —cuenta este

término, busca este patrón, devuelve este contexto, compara estas versiones—, y cuanto mejor formulada esté la pregunta, más útil será la respuesta.

1.13. Funciones, informes y trazabilidad

En programación, una función permite encapsular una tarea: recibe datos, aplica una operación y puede devolver un resultado. En una revisión lingüística, las funciones aportan orden y permiten reconstruir lo hecho, porque en lugar de copiar el mismo código varias veces se crea una comprobación reutilizable: una función para contar términos, otra para detectar espacios dobles, otra para buscar patrones de puntuación, otra para generar un resumen.

La trazabilidad pesa mucho en los entornos profesionales. Si una revisora pregunta cómo se detectaron los casos, el equipo debe poder responder que se usó este patrón, sobre estos textos y con esta normalización, y si el cliente pide repetir la comprobación en una versión nueva, la función permite hacerlo sin rediseñar todo el proceso.

El informe final es la forma comunicativa de esa trazabilidad. No basta con que el código funcione: el resultado debe ser comprensible para una persona que toma decisiones, y un informe útil explica qué se ha buscado, qué se ha encontrado, dónde aparece y qué cautela interpretativa debe aplicarse, además de listar números.

En Verba Pública, el informe no pretende reemplazar una revisión de estilo completa; su finalidad es preparar una primera lectura asistida que reduzca la incertidumbre, ordene prioridades y documente indicios. Ese es un uso profesional realista de Python en lingüística aplicada.

1.14. Límites de los métodos basados en reglas

Los métodos basados en reglas son transparentes, fáciles de enseñar y útiles para fenómenos con forma reconocible, pero tienen límites. El lenguaje natural es variable, ambiguo y dependiente del contexto: una palabra puede pertenecer a varias categorías; una construcción puede ser aceptable en un contexto y discutible en otro, y un sinónimo puede ser deseable o problemático según la política terminológica.

Frente a los modelos estadísticos o neuronales, las reglas simples no aprenden de datos ni generalizan semánticamente, una limitación que en ciertos encargos se convierte en ventaja: si se necesita comprobar una convención formal muy concreta, una regla expresa puede ser más adecuada que un sistema opaco. En PLN profesional conviven enfoques distintos: reglas, recursos lingüísticos, corpus, aprendizaje automático y revisión humana.

La unidad se centra deliberadamente en reglas y operaciones básicas porque construyen una base conceptual bien asentada. Antes de usar bibliotecas avanzadas, conviene entender qué significa leer un texto, dividirlo, recorrerlo, contar, buscar y devolver resultados, fundamentos que seguirán siendo necesarios cuando el alumnado trabaje con herramientas más complejas.

2. Análisis automático de textos para revisión lingüística con Python

2.1. Leer textos como datos

Para que Python pueda analizar un documento, primero debe leerlo como texto. En la práctica, eso significa abrir un archivo y convertir su contenido en una cadena de caracteres. La cadena es el tipo de dato básico para trabajar con texto: permite buscar palabras, contar secuencias, dividir por signos, extraer fragmentos y aplicar expresiones regulares.

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

Un detalle técnico importante es la codificación. En español aparecen tildes, eñes, comillas angulares y otros caracteres que deben interpretarse correctamente, y por eso se indica `encoding="utf-8"` (Python Software Foundation, 2026a). UTF-8 es una codificación estándar, muy extendida, que permite representar caracteres de muchas lenguas; si se omite, el sistema puede elegir una por defecto y producir errores o caracteres dañados.

```
with open("texto1.txt", encoding="utf-8") as archivo:
    texto1 = archivo.read()
```

El bloque `with` abre el archivo y se encarga de cerrarlo al terminar. La variable `texto1` guarda todo el contenido como una cadena, y desde ese momento el texto ya puede tratarse como dato lingüístico procesable.

El formato más cómodo para empezar es el `.txt`, que contiene texto plano, es decir, caracteres sin estilos, sin tablas internas y sin formatos complejos. Por eso resulta muy útil para aprender PLN: reduce el ruido técnico y permite concentrarse en el contenido lingüístico.

Una forma sencilla de leer un archivo es:

```
texto_a = open("version_a.txt", encoding="utf-8").read()
texto_b = open("version_b.txt", encoding="utf-8").read()
```

Aquí ocurren varias cosas importantes:

- `open(...)` abre el archivo.
- `"version_a.txt"` indica el nombre del archivo.
- `encoding="utf-8"` le dice a Python cómo interpretar caracteres como á, ñ, ü o ¿.
- `.read()` lee todo el contenido y lo convierte en una cadena de texto.
- `texto_a` y `texto_b` son variables que guardan cada versión.

En español, la codificación pesa mucho. Si se lee mal un archivo, pueden aparecer caracteres extraños, como `implementaciÃ³n` en lugar de `implementación`; el problema no es lingüístico, pero afecta directamente al análisis, porque si Python no reconoce bien las tildes los conteos pueden fallar.

2.2. Rutas de archivos y `pathlib`

Cuando los documentos están en una carpeta local, conviene trabajar con rutas claras. El módulo `pathlib` permite representar rutas de forma más legible y menos dependiente de los detalles del sistema operativo (Python Software Foundation, 2026b): en lugar de escribir una ruta como una cadena larga, se crea un objeto `Path`.

```
from pathlib import Path

carpeta_trabajo = Path("ruta/a/la/carpeta/de/textos")

ruta_a = carpeta_trabajo / "version_a.txt"
ruta_b = carpeta_trabajo / "version_b.txt"

with open(ruta_a, encoding="utf-8") as f:
    texto_a = f.read()

with open(ruta_b, encoding="utf-8") as f:
    texto_b = f.read()
```

Esta forma tiene dos ventajas. La primera es que separa la carpeta de trabajo del nombre de cada archivo. La segunda es que usa `with open(...)` as `f`, una estructura que abre el archivo, permite leerlo y lo cierra automáticamente al terminar.

Desde el punto de vista del PLN, lo relevante es que cada documento queda representado como un **string**, es decir, una cadena de caracteres. Para Python, un informe institucional, una frase corta o un discurso entero son objetos manipulables siempre que estén guardados como texto.

2.3. Cadenas, listas y corpus pequeño

Cuando solo hay dos versiones, es tentador crear variables separadas y repetir el mismo código para cada una:

```
texto_a_lower = texto_a.lower()
texto_b_lower = texto_b.lower()
```

Esto funciona, pero escala mal: si mañana llegan cincuenta versiones, repetir cincuenta líneas parecería resultar incómodo y propenso a errores.

Una idea básica en programación es agrupar los elementos relacionados en una **lista**:

```
textos = [texto_a, texto_b]
```

La lista `textos` contiene todas las versiones que queremos analizar. A partir de ese momento, podemos recorrerlas con un bucle:

```
for i, texto in enumerate(textos, start=1):
    print(f"Analizando versión {i}")
```

La función `enumerate()` permite recorrer una lista y, al mismo tiempo, obtener un número de orden. El argumento `start=1` indica que queremos empezar a contar en 1 y no en 0, algo más natural en los informes destinados a personas: **versión 1**, **versión 2**, **versión 3**.

Esta estructura cambia la forma de pensar el análisis: la solución deja de servir para un archivo concreto y pasa a servir para cualquier conjunto de textos:

```
textos = [texto_a, texto_b, texto_c]

for i, texto in enumerate(textos, start=1):
    longitud = len(texto)
    print(f"Versión {i}: {longitud} caracteres")
```

En proyectos de revisión, esta generalización es muy valiosa. La misma lógica sirve para dos versiones de una traducción, tres discursos institucionales o cien documentos de una licitación.

2.4. Segmentar en oraciones con split y strip

La oración es una unidad lingüística central. En revisión automática, segmentar un texto en oraciones permite observar el contenido en fragmentos manejables, porque no es lo mismo detectar que una palabra aparece en un documento entero que saber en qué oración aparece.

Una estrategia inicial consiste en dividir el texto cada vez que aparece un punto:

```
oraciones_a = texto_a.split(".")
oraciones_b = texto_b.split(".")
```

El método `.split(".")` corta la cadena cada vez que encuentra el carácter ".". El resultado es una lista:

```
for i, oracion in enumerate(oraciones_a[:3], start=1):
    print(f"{i}. {oracion.strip()}")
```

El método `.strip()` elimina los espacios al principio y al final del fragmento, algo útil porque, después de cortar por puntos, muchas oraciones empiezan con un espacio.

Esta segmentación es sencilla y comprensible, pero no perfecta, porque en español el punto no siempre marca el final de una oración: puede aparecer en abreviaturas, cifras, iniciales, direcciones web o enumeraciones:

Tabla 9. *Puntos que no cierran oración*

Caso	Problema
Sr. García	El punto no cierra una oración
art. 15	El punto pertenece a una abreviatura
3.2 millones	El punto puede formar parte de una cifra o sección
www.ejemplo.org	El punto aparece dentro de una dirección

Para un primer análisis de control de calidad, `split(".")` puede ser suficiente. Para herramientas profesionales más precisas, convendría usar segmentadores especializados, como los que ofrecen bibliotecas de PLN.

Lo importante es entender la decisión lingüística que hay detrás: segmentar es elegir qué cuenta como unidad de análisis.

2.5. Contar palabras y normalizar con minúsculas

Contar palabras es una de las operaciones más simples y, al mismo tiempo, más útiles del análisis textual. En un contexto de revisión, la frecuencia permite detectar preferencias terminológicas, cambios de estilo o posibles incoherencias.

Supongamos que queremos saber cuántas veces aparece la palabra *implementación* en dos versiones:

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

```
texto_a_lower = texto_a.lower()
texto_b_lower = texto_b.lower()

conteo_a = texto_a_lower.count("implementación")
conteo_b = texto_b_lower.count("implementación")

print(f"Versión A: {conteo_a} apariciones")
print(f"Versión B: {conteo_b} apariciones")
```

El método `.lower()` convierte todo el texto a minúsculas, de modo que *Implementación* al inicio de una oración e *implementación* en medio de una frase se cuentan igual.

Sin esta normalización, Python distinguiría entre:

- Implementación
- implementación
- IMPLEMENTACIÓN

Desde el punto de vista lingüístico, pueden ser la misma unidad léxica; desde el punto de vista computacional, son cadenas distintas. Normalizar el texto permite acercar la representación computacional a la pregunta lingüística.

Ahora bien, `.count("implementación")` tiene límites: cuenta las apariciones exactas de esa secuencia de caracteres, de modo que no agrupa automáticamente variantes como:

- implementar
- implementado
- implementaciones
- implementativa

Para agrupar formas relacionadas haría falta lematización o análisis morfológico. En esta unidad trabajamos con conteo exacto porque es transparente y fácil de interpretar.

2.6. Recorrer varias versiones con `enumerate`

El análisis mejora cuando dejamos de pensar en una pareja fija de textos y empezamos a pensar en un conjunto variable de versiones:

```
textos = [texto_a, texto_b, texto_c]
palabra = "implementación"

for i, texto in enumerate(textos, start=1):
    texto_lower = texto.lower()
    conteo = texto_lower.count(palabra)
    print(f"Versión {i}: {conteo} apariciones de '{palabra}'")
```

Esta estructura contiene tres ideas esenciales:

- `textos` agrupa todos los documentos.
- `for` repite el mismo análisis para cada documento.
- `enumerate()` permite etiquetar cada salida con un número de versión.

El resultado puede servir para detectar diferencias llamativas:

Tabla 10. Apariciones de implementación por versión

Versión	Apariciones de implementación	Interpretación inicial
1	8	Alta preferencia por el término
2	1	Posible uso de alternativa terminológica
3	0	Puede haber sustitución por otro concepto o paráfrasis

La interpretación no debe precipitarse. Que una versión use menos un término no la hace necesariamente peor: puede ser más concisa, haber reformulado el contenido o emplear un término recomendado. El conteo solo abre una pregunta.

2.7. Detectar ausencia de espacio después de puntuación

Uno de los problemas formales más fáciles de detectar automáticamente es la ausencia de espacio después de punto o coma:

Tabla 11. Ausencia de espacio tras punto o coma

Fragmento	Problema
El informe fue aprobado.La comisión lo publicó.	Falta espacio tras punto
El proyecto incluye formación,seguimiento y evaluación.	Falta espacio tras coma

Estos errores parecen pequeños, pero en los documentos institucionales afectan a la calidad percibida y pueden dificultar procesos automáticos posteriores, como la segmentación o la tokenización.

Para detectarlos, se puede usar una expresión regular:

```
import re

errores = [m.start() for m in re.finditer(r"[.,][^\s]", texto_a)]
print(f"Número de signos sin espacio después: {len(errores)}")
```

La expresión `r"[.,][^\s]"` se lee así:

Tabla 12. Lectura del patrón `r"[.,][^\s]"`

Parte	Significado
<code>r"..."</code>	Cadena cruda (<i>raw</i>), útil para escribir patrones de expresiones regulares

[.,]	Un punto o una coma
[^\s]	Un carácter que no sea espacio

Por tanto, el patrón busca un punto o una coma seguidos inmediatamente de un carácter que no sea espacio.

El método `re.finditer()` devuelve coincidencias con información de posición. Por eso podemos usar `m.start()` para saber en qué índice del texto empieza cada caso.

2.8. Recuperar contexto con índices y *slicing*

Un informe que solo dice «hay 12 errores» se queda corto, porque la persona que revisa necesita saber dónde están. Para eso se extraen fragmentos de contexto:

```
for pos in errores:
    inicio = max(pos - 20, 0)
    fin = pos + 20
    fragmento = texto_a[inicio:fin]
    print(f"...{fragmento}...")
```

Este fragmento introduce varias ideas:

- `pos` es la posición donde se detectó el patrón.
- `pos - 20` toma veinte caracteres anteriores.
- `pos + 20` toma veinte caracteres posteriores.
- `max(pos - 20, 0)` evita que el índice inicial sea negativo.
- `texto_a[inicio:fin]` extrae una porción del texto.

El resultado permite ver algo como:

```
...evaluación.La comisión aprobó...
...formación,seguimiento y apoyo...
```

La extracción de contexto es una práctica habitual en la revisión automática: no basta con encontrar el error, hay que mostrarlo de forma útil.

2.9. Localizar dobles espacios

Los dobles espacios son otro problema formal frecuente. Suelen aparecer al editar, copiar y pegar o combinar fragmentos procedentes de distintas fuentes:

```
La institución  publicará el informe final.
```

Una detección mínima puede hacerse con el operador `in`:

```
hay_dobles_espacios = "  " in texto_a
print(hay_dobles_espacios)
```

La expresión " " in texto_a pregunta si la secuencia de dos espacios aparece en el texto. Devuelve True o False.

Para contar cuántas veces aparece:

```
total = texto_a.count(" ")
print(f"Dobles espacios detectados: {total}")
```

Y para saber en qué oraciones se encuentran:

```
dobles = [
    i + 1
    for i, oracion in enumerate(oraciones_a)
    if " " in oracion
]

print(f"Aparecen en las oraciones: {dobles}")
```

Esta estructura es una **lista por comprensión**, que construye una lista nueva a partir de una condición; en este caso, guarda los números de las oraciones que contienen dobles espacios.

La parte i + 1 se usa porque Python empieza a contar en 0, pero en un informe para personas suele ser más natural empezar por 1.

2.10. Comparación terminológica básica

Cuando se trabaja con varios textos, también conviene guardar las oraciones de cada texto en una lista:

```
textos = [texto_a, texto_b, texto_c]
oraciones_textos = [oraciones_a, oraciones_b, oraciones_c]
```

Para recorrer al mismo tiempo cada texto y su lista de oraciones, se puede usar zip():

```
for i, (texto, oraciones) in enumerate(zip(textos, oraciones_textos), start=1):
    dobles = [
        n + 1
        for n, oracion in enumerate(oraciones)
        if " " in oracion
    ]
    print(f"Versión {i}: {texto.count(' ')} dobles espacios")
    print(f"Oraciones afectadas: {dobles}")
```

zip(textos, oraciones_textos) empareja los elementos de ambas listas:

Tabla 13. *Emparejamiento de textos y oraciones con zip()*

Elemento de textos	Elemento de oraciones_textos
texto_a	oraciones_a

texto_b	oraciones_b
texto_c	oraciones_c

Esta técnica es muy útil cuando varias estructuras de datos representan distintos niveles del mismo corpus: texto completo, oraciones, metadatos, proveedor, fecha de entrega o estado de revisión.

La comparación terminológica es el núcleo profesional de esta unidad, porque en los textos paralelos monolingües muchas divergencias responden a diferencias de elección léxica más que a errores gramaticales.

Supongamos que queremos comparar estas parejas:

```
parejas_terminos = [
    ("equipo", "grupo"),
    ("compañía", "empresa"),
    ("gerente", "director"),
    ("procesos", "procedimientos"),
]
```

Cada pareja representa dos formas que pueden competir dentro de una misma política terminológica: una institución puede considerar que ambas son aceptables en contextos distintos o puede preferir una.

Un análisis sencillo cuenta cada término en cada versión:

```
texto_a_lower = texto_a.lower()
texto_b_lower = texto_b.lower()

for termino_1, termino_2 in parejas_terminos:
    c1_a = texto_a_lower.count(termino_1)
    c2_a = texto_a_lower.count(termino_2)
    c1_b = texto_b_lower.count(termino_1)
    c2_b = texto_b_lower.count(termino_2)

    print(f"{termino_1}: versión A={c1_a}, versión B={c1_b}")
    print(f"{termino_2}: versión A={c2_a}, versión B={c2_b}")
```

El resultado puede revelar situaciones como estas:

Tabla 14. Situaciones terminológicas y posibles interpretaciones

Situación	Posible interpretación
Una versión usa <i>equipo</i> y otra <i>grupo</i>	Divergencia terminológica entre proveedores
Una versión alterna <i>empresa</i> y <i>compañía</i>	Falta de uniformidad interna
Un término recomendado no aparece	Posible incumplimiento de glosario
Un término aparece en exceso	Puede haber traducción literal, repetición o preferencia estilística

La tabla no decide por sí sola qué término es mejor, pero proporciona evidencia para discutirlo.

2.11. Tablas de resultados

En revisión profesional, la forma de presentar los resultados importa, porque una salida desordenada puede ser difícil de interpretar. Python permite alinear texto y números mediante cadenas formateadas.

```
print(f"{'Término':<20} | {'Versión 1':^10} | {'Versión 2':^10}")
print("-" * 48)

for termino_1, termino_2 in parejas_terminos:
    print(f"{termino_1:<20} | {c1_a:^10} | {c1_b:^10}")
    print(f"{termino_2:<20} | {c2_a:^10} | {c2_b:^10}")
    print("-" * 48)
```

Las expresiones entre llaves controlan el formato:

Tabla 15. Especificadores de formato en cadenas formateadas

Código	Significado
{termino_1:<20}	Escribe el término alineado a la izquierda en 20 caracteres
{c1_a:^10}	Centra el número en 10 caracteres
"-" * 48	Repite el guion 48 veces para crear una línea separadora

Este tipo de formato va más allá de lo estético: ayuda a leer la comparación como una tabla, y cuando los resultados se entregan a un equipo editorial, la claridad visual y la precisión técnica van juntas.

2.12. Comparar términos en muchas versiones

La comparación puede generalizarse a cualquier número de textos:

```
textos = [texto_a, texto_b, texto_c]
textos_lower = [texto.lower() for texto in textos]

cabecera = "Término".ljust(20)
for i in range(len(textos_lower)):
    cabecera += f" | Versión {i + 1:^8}"
print(cabecera)

for pareja in parejas_terminos:
    for termino in pareja:
        fila = f"{termino:<20}"
        for texto in textos_lower:
            fila += f" | {texto.count(termino):^10}"
        print(fila)
```

Aquí aparece una nueva lista por comprensión:

```
textos_lower = [texto.lower() for texto in textos]
```

Esta línea crea una lista con todas las versiones en minúsculas, una forma compacta de preparar los datos antes de contarlos.

La comparación en muchas versiones permite detectar patrones más ricos:

Tabla 16. *Frecuencia de términos en tres versiones*

Término	Versión 1	Versión 2	Versión 3
adquirir	4	0	1
obtener	0	5	2
conseguir	1	0	6
trabajador	8	1	0
empleado	0	6	2
asalariado	0	0	5

Una tabla así invita a formular hipótesis:

- Quizá cada proveedor tiene una preferencia léxica estable.
- Quizá una versión es más técnica y otra más general.
- Quizá se están usando términos que no pertenecen al mismo registro.
- Quizá hay que consolidar un glosario antes de publicar.

El análisis cuantitativo no cierra la interpretación; la abre de forma ordenada.

2.13. Posibles galicismos: buscar candidatos *a* + infinitivo

En español normativo, una construcción como *temas a tratar* suele citarse como posible calco del francés cuando funciona como sustantivo + *a* + infinitivo, y en muchos contextos se recomienda reformularla con estructuras como estas:

Tabla 17. *Construcciones con *a* + infinitivo y posibles alternativas*

Construcción revisable	Posible alternativa
asuntos a debatir	asuntos que se deben debatir
tareas a realizar	tareas que hay que realizar
documentos a presentar	documentos que deben presentarse

No obstante, el diagnóstico no puede automatizarse con una regla tan simple como «toda secuencia *a* + infinitivo es incorrecta», porque en español hay muchísimas construcciones correctas de este tipo:

- vamos a revisar
- empezó a trabajar

- aprendió a programar
- ayuda a entender

Por eso conviene hablar de **posibles galicismos** o **candidatos a revisión**, y no de errores automáticos.

Un patrón inicial puede localizar secuencias de palabra + a + infinitivo:

```
import re

patron = r"(\w+)\s+a\s+(\w+(ar|er|ir))\b"
resultados = re.findall(patron, texto_a.lower())

pares = [(anterior, verbo) for anterior, verbo, _ in resultados]

for anterior, verbo in pares:
    print(f"{anterior} a {verbo}")
```

La lista pares conserva solo la palabra anterior y el verbo; el tercer elemento de cada coincidencia, la terminación (*ar*, *er* o *ir*), se descarta con `_` porque no interesa para el informe.

Este análisis es útil como filtro inicial: si aparecen muchos casos, la lingüista puede revisar el contexto y distinguir entre ellos:

Tabla 18. *Casos de a + infinitivo detectados y lectura lingüística*

Caso detectado	Lectura posible
documentos a presentar	Candidato a revisión
vamos a presentar	Construcción verbal correcta
ayuda a presentar	Régimen verbal correcto
aspectos a considerar	Candidato a revisión según estilo

Para automatizar mejor esta tarea haría falta combinar varias técnicas: tokenización, lematización y etiquetado morfosintáctico. En particular, habría que comprobar si la palabra anterior es un sustantivo y si la secuencia forma realmente una construcción nominal.

2.14. Buscar gerundios y contexto inmediato

El gerundio en español tiene usos correctos y usos problemáticos. Puede expresar simultaneidad, modo o proceso:

- El equipo revisó el informe siguiendo las instrucciones.
- La comisión analizó los datos comparando las dos versiones.

Pero también puede aparecer en usos desaconsejados, como el gerundio de posterioridad o ciertos gerundios especificativos:

Tabla 19. *Gerundios desaconsejados*

Tipo	Fragmento revisable	Motivo
Posterioridad	Se aprobó el informe, publicándose al día siguiente.	El gerundio expresa una acción posterior

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

Especificativo	Se envió una carta informando del cambio.	Puede funcionar como modificador poco natural
----------------	---	---

Una regla formal inicial consiste en buscar palabras terminadas en *-ando*, *-iendo* o *-yendo*:

```
import re

patron = r"(\w+)?\s*(\w+(ando|iendo|yendo))\s*(\w+)?"
coincidencias = re.findall(patron, texto_a.lower())

gerundios = [
    (anterior if anterior else "", gerundio, siguiente if siguiente else "")
    for anterior, gerundio, _, siguiente in coincidencias
]

for anterior, gerundio, siguiente in gerundios:
    print(anterior, gerundio, siguiente)
```

Este patrón busca el gerundio y captura, si existen, una palabra anterior y una palabra siguiente. El contexto mínimo ayuda a decidir si el caso requiere revisión.

De nuevo, la herramienta no declara que un gerundio sea incorrecto; solo localiza formas que una persona puede evaluar. La diferencia entre detectar y diagnosticar es esencial:

Tabla 20. Detección automática frente a diagnóstico lingüístico

Detección automática	Diagnóstico lingüístico
Encuentra palabras terminadas en <i>-ando</i> , <i>-iendo</i> o <i>-yendo</i>	Decide si el uso del gerundio es aceptable
Muestra contexto inmediato	Interpreta relación temporal, sintáctica y semántica
Cuenta ocurrencias	Valora si hay patrón estilístico problemático

Como se explicó en el marco teórico, un **falso positivo** es un caso que la herramienta marca como sospechoso, pero que no constituye realmente un problema.

En análisis de revisión lingüística, los falsos positivos son inevitables cuando usamos reglas simples. Por ejemplo:

```
patron = r"(\w+)\s+a\s+(\w+(ar|er|ir))\b"
```

Este patrón puede detectar:

- vamos a revisar
- ayuda a entender
- documentos a presentar

Solo algunos casos serán revisables como posible calco, porque el patrón no sabe si la palabra anterior es un sustantivo, un verbo conjugado o una forma auxiliar, ni conoce el régimen verbal.

Esto no invalida el análisis; al contrario, enseña una lección metodológica importante. En PLN, muchas herramientas funcionan por aproximación, y una buena herramienta es la que reduce el trabajo humano sin ocultar sus límites, más que la que no se equivoca nunca.

Para una lingüista, el objetivo es aprender a preguntar, en lugar de aceptar ciegamente la salida del programa:

- ¿Qué está detectando exactamente esta regla?
- ¿Qué casos deja fuera?
- ¿Qué casos marca indebidamente?
- ¿Qué conocimiento lingüístico haría falta añadir?
- ¿Qué coste tendría mejorar la precisión?

Esta actitud vigilante distingue el uso superficial de Python de un verdadero análisis lingüístico computacional.

2.15. Organizar el análisis con funciones

Cuando un análisis crece, conviene organizarlo en **funciones**. Una función es un bloque de código con nombre que realiza una tarea concreta.

Por ejemplo:

```
def contar_palabra(texto, palabra):
    texto_lower = texto.lower()
    return texto_lower.count(palabra)
```

Esta función recibe un texto y una palabra, convierte el texto a minúsculas y devuelve el número de apariciones.

Podemos usarla así:

```
conteo = contar_palabra(texto_a, "implementación")
print(conteo)
```

Las funciones aportan varias ventajas:

Tabla 21. Ventajas de organizar el análisis en funciones

Ventaja	Explicación
Organización	Cada tarea tiene un nombre claro
Reutilización	La misma función sirve para otros textos
Legibilidad	El informe final puede llamar funciones en orden
Mantenimiento	Si se mejora una regla, se cambia en un solo lugar

Escalabilidad	Es más fácil añadir nuevas comprobaciones
---------------	---

Sin funciones, el código puede convertirse en una sucesión larga de instrucciones repetidas. Con funciones, el análisis se parece más a un conjunto ordenado de herramientas.

Un sistema básico de revisión puede organizarse en funciones como estas:

```
def contar_palabra(texto, palabra):
    return texto.lower().count(palabra)
```

```
def buscar_falta_espacio(texto):
    import re
    return [m.start() for m in re.finditer(r"[.,][^\s]", texto)]
```

```
def detectar_dobles_espacios(texto, oraciones):
    posiciones = [
        i + 1
        for i, oracion in enumerate(oraciones)
        if " " in oracion
    ]
    return texto.count(" "), posiciones
```

```
def analizar_a_infinitivo(texto):
    import re
    patron = r"(\w+)\s+a\s+(\w+(ar|er|ir))\b"
    resultados = re.findall(patron, texto.lower())
    return [(anterior, verbo) for anterior, verbo, _ in resultados]
```

```
def buscar_gerundios(texto):
    import re
    patron = r"(\w+)?\s*(\w+(ando|iendo|yendo))\s*(\w+)?"
    coincidencias = re.findall(patron, texto.lower())
    return [
        (anterior if anterior else "", gerundio, siguiente if siguiente else "")
        for anterior, gerundio, _, siguiente in coincidencias
    ]
```

Cada función resuelve una pregunta concreta:

Tabla 22. Funciones del sistema de revisión y preguntas que responden

Función	Pregunta que responde
contar_palabra()	¿Cuántas veces aparece este término?
buscar_falta_espacio()	¿Dónde falta espacio tras punto o coma?
detectar_dobles_espacios()	¿Cuántos dobles espacios hay y en qué oraciones?
analizar_a_infinitivo()	¿Qué secuencias pueden requerir revisión como a + infinitivo?
buscar_gerundios()	¿Qué gerundios aparecen y con qué contexto

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

	mínimo?
--	---------

El nombre de una función debe ser transparente. Una persona que lea `buscar_gerundios(texto)` entiende la intención sin tener que estudiar cada línea interna.

2.16. Devolver resultados con `return`

Una función puede imprimir algo en pantalla o devolver un resultado, y no es lo mismo una cosa que otra.

Observa esta función:

```
def sumar(a, b):  
    resultado = a + b
```

Calcula `a + b`, pero no devuelve el resultado. Si se intenta guardar la salida:

```
x = sumar(3, 4)  
print(x)
```

Python imprimirá:

```
None
```

None significa que la función no ha devuelto ningún valor.

Para que el resultado pueda usarse fuera de la función, necesitamos `return`:

```
def sumar(a, b):  
    resultado = a + b  
    return resultado
```

O, de forma más breve:

```
def sumar(a, b):  
    return a + b
```

Entonces:

```
x = sumar(3, 4)  
print(x)
```

produce:

```
7
```


En un informe de revisión lingüística, return es importante porque permite combinar resultados. Una función puede contar términos y devolver números; otra puede detectar errores y devolver posiciones; otra puede construir una tabla. Si solo imprimen, los resultados se ven en pantalla, pero es más difícil reutilizarlos.

2.17. Generar un informe final

El objetivo práctico es producir un informe ordenado, más que acumular fragmentos de código. Un informe final puede organizarse por secciones:

- Frecuencia de términos clave.
- Ausencias de espacio tras puntuación.
- Dobles espacios.
- Comparación terminológica.
- Posibles estructuras *a* + infinitivo.
- Gerundios localizados.
- Observaciones para revisión humana.

Un esquema simplificado podría ser:

```
textos = [texto_a, texto_b]
oraciones_textos = [oraciones_a, oraciones_b]

print("INFORME DE REVISIÓN LINGÜÍSTICA")
print("=" * 40)

for i, texto in enumerate(textos, start=1):
    print(f"\nVERSIÓN {i}")
    print("-" * 20)

    conteo = contar_palabra(texto, "implementación")
    print(f"Frecuencia de 'implementación': {conteo}")

    errores = buscar_falta_espacio(texto)
    print(f"Signos sin espacio después: {len(errores)}")

    total_dobles, oraciones = detectar_dobles_espacios(
        texto,
        oraciones_textos[i - 1]
    )
    print(f"Dobles espacios: {total_dobles}")
    print(f"Oraciones afectadas: {oraciones}")

    candidatos = analizar_a_infinitivo(texto)
    print(f"Posibles casos de 'a + infinitivo': {len(candidatos)}")

    gerundios = buscar_gerundios(texto)
    print(f"Gerundios encontrados: {len(gerundios)}")
```

Este informe no pretende ser el documento final que se entregaría a una institución; es una base a la que se podrían añadir:

- Exportación a .csv o .xlsx.
- Resúmenes por proveedor.
- Fragmentos de contexto para cada hallazgo.
- Clasificación manual de casos correctos e incorrectos.
- Gráficos de distribución terminológica.
- Integración con glosarios institucionales.

La idea de fondo es que Python organiza la evidencia, mientras que la decisión profesional sigue perteneciendo a la especialista.

2.18. Cómo leer los resultados antes de corregir

Una buena práctica consiste en leer los resultados con tres preguntas:

- ¿Qué regla produjo este hallazgo?
- ¿Qué parte del texto se muestra como contexto?
- ¿Qué decisión humana hace falta ahora?

Estas preguntas evitan una confianza excesiva en la salida automática y ayudan a documentar el trabajo: si una incidencia se descarta, puede anotarse por qué, y si se confirma, puede corregirse con una justificación. En entornos profesionales, esta trazabilidad puede pesar tanto como la corrección misma.

Reglas simples frente a análisis lingüístico avanzado

Las técnicas de esta unidad se basan en reglas simples:

- Cortar por puntos.
- Contar cadenas exactas.
- Buscar patrones con expresiones regulares.
- Identificar terminaciones como *-ando*, *-iendo* o *-yendo*.

Estas reglas son transparentes, porque podemos explicar exactamente qué hacen, y esa transparencia es una virtud en la formación y en la revisión profesional. Aun así, también tienen límites:

Tabla 23. Reglas simples y sus límites

Regla simple	Límite
<code>split(". ")</code>	Confunde puntos de abreviaturas con finales de oración
<code>.count("empresa")</code>	Puede contar partes de palabras o no agrupar variantes flexivas
Patrón <i>a</i> + infinitivo	No distingue sustantivos de verbos conjugados
Terminaciones de gerundio	No interpreta la función sintáctica del gerundio
Conteo de sinónimos	No sabe si dos términos son equivalentes en contexto

Para mejorar la precisión, se pueden incorporar técnicas más avanzadas:

- **Tokenización**, para separar el texto en unidades lingüísticas más precisas.
- **Lematización**, para agrupar formas como *trabaja*, *trabajó* y *trabajar*.
- **Etiquetado morfosintáctico**, para saber si una palabra es sustantivo, verbo, adjetivo, etc.
- **Análisis de dependencias**, para estudiar relaciones sintácticas.
- **Reconocimiento de entidades**, para detectar instituciones, lugares o personas.
- **Modelos estadísticos o neuronales**, para capturar patrones no formulados manualmente.

La progresión natural en PLN consiste en empezar por reglas simples, comprender sus límites y avanzar hacia herramientas más sofisticadas cuando la pregunta lo requiere.

Buenas prácticas para revisar textos con Python

Un análisis automático de revisión lingüística debe ser claro, reproducible y prudente. Algunas buenas prácticas son:

1. Nombrar bien las variables. `texto_a` es más claro que `x`.
2. Guardar juntos los textos relacionados. Una lista facilita el análisis conjunto.
3. Normalizar cuando sea necesario. `.lower()` evita diferencias irrelevantes entre mayúsculas y minúsculas.
4. Conservar contexto. Un hallazgo sin fragmento puede ser difícil de corregir.
5. Distinguir detección de diagnóstico: que una regla encuentre algo no significa que sea un error.
6. Presentar resultados con etiquetas claras. El informe debe poder leerlo otra persona.
7. Evitar copiar y pegar bloques repetidos. Las funciones reducen errores y mejoran la reutilización.
8. Documentar los límites. Toda herramienta debe decir qué detecta y qué no.
9. Mantener el criterio lingüístico en el centro: Python apoya la revisión, pero no sustituye la interpretación.

Estas prácticas importan tanto como la sintaxis. En entornos profesionales, un análisis útil es el que permite tomar decisiones con confianza, y no solo el que produce números.

Errores frecuentes de aprendizaje

- Confundir una coincidencia automática con un error lingüístico confirmado.
- Pensar que `split(" ")` segmenta siempre las oraciones correctamente.
- Olvidar `encoding="utf-8"` al leer textos con tildes, eñes o comillas españolas.
- Contar términos sin normalizar mayúsculas y minúsculas cuando la comparación lo requiere.
- Usar `count` sin recordar que busca secuencias, no necesariamente palabras completas.
- Interpretar una divergencia terminológica como error sin consultar el glosario o el contexto.
- Creer que una expresión regular comprende gramática, régimen verbal o estilo.
- Diseñar patrones demasiado amplios y no prever falsos positivos.
- Diseñar patrones demasiado estrechos y dejar fuera casos relevantes.
- Usar `print` en todo el código y luego no poder reutilizar los resultados.
- Repetir bloques casi idénticos en lugar de crear funciones.
- Presentar un informe sin contexto suficiente para que una revisora pueda decidir.

Síntesis final

Esta unidad muestra cómo Python puede apoyar la revisión lingüística de textos paralelos monolingües. Más que automatizar el juicio experto, se busca construir una cadena de análisis que convierta los textos en datos observables: se leen archivos, se guardan versiones, se segmentan oraciones, se cuentan términos, se buscan patrones formales y se generan resultados interpretables.

El marco teórico de la unidad enlaza tres ámbitos: la revisión y evaluación de la calidad en servicios lingüísticos, la lingüística de corpus y el PLN basado en reglas. De la revisión profesional se toma la importancia del criterio y de la trazabilidad; de la lingüística de corpus, la idea de observar patrones en colecciones de textos, y del PLN, herramientas como cadenas, listas, bucles, expresiones regulares y funciones.

El aprendizaje más importante es metodológico: el código debe formular preguntas concretas. Python puede ayudar a responder «cuántas veces aparece este término», «dónde hay una coma sin espacio posterior» o «qué segmentos contienen una forma terminada en gerundio», pero no puede responder por sí solo «qué versión es mejor» sin un marco de decisión humano.

Para una alumna o un alumno lingüista, esta unidad ofrece una base profesional muy valiosa, porque permite pasar de la lectura lineal a la lectura asistida por evidencias, y de la intuición individual a un procedimiento más documentable, una competencia que pesa mucho en el PLN aplicado a la revisión, la traducción, la edición y el control de calidad textual.

Cierre narrativo

Al final de la simulación, Verba Pública entrega a la institución un informe preliminar de revisión, y no una sentencia automática: términos que conviene unificar, patrones formales que deben comprobarse, estructuras candidatas a lectura lingüística y una explicación clara de las cautelas. La revisora sénior conserva la última palabra, pero ya no parte de una página en blanco.

La institución gana tiempo y transparencia, y el equipo lingüístico, una forma de justificar mejor sus recomendaciones. Por su parte, el alumnado aprende que programar para PLN consiste en diseñar procedimientos que respeten la complejidad del lenguaje y ayuden a tomar mejores decisiones, más allá de escribir código que funcione.

Referencias

- Bird, S., Klein, E. y Loper, E. (2009). *Natural language processing with Python: Analyzing text with the Natural Language Toolkit*. O'Reilly Media. <https://www.nltk.org/book/>
- Bowker, L. (2002). *Computer-aided translation technology: A practical introduction*. University of Ottawa Press.
- Friedl, J. E. F. (2006). *Mastering regular expressions* (3.ª ed.). O'Reilly Media.
- International Organization for Standardization. (2015). *Translation services: Requirements for translation services* (ISO Standard No. 17100:2015). <https://www.iso.org/standard/59149.html>
- International Organization for Standardization. (2024). *Translation services: Evaluation of translation output. General guidance* (ISO Standard No. 5060:2024). <https://www.iso.org/standard/80786.html>
- Jurafsky, D. y Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3.ª ed., borrador en línea del 19 de agosto de 2026). <https://web.stanford.edu/~jurafsky/slp3/>

- Kiss, T. y Strunk, J. (2006). Unsupervised multilingual sentence boundary detection. *Computational Linguistics*, 32(4), 485–525. <https://doi.org/10.1162/coli.2006.32.4.485>
- Lommel, A., Uszkoreit, H. y Burchardt, A. (2014). Multidimensional Quality Metrics (MQM): A framework for declaring and describing translation quality metrics. *Tradumática*, (12), 455–463. <https://doi.org/10.5565/rev/tradumatica.77>
- McEnery, T. y Hardie, A. (2012). *Corpus linguistics: Method, theory and practice*. Cambridge University Press.
- Olohan, M. (2004). *Introducing corpora in translation studies*. Routledge.
- Python Software Foundation. (2026a). *Built-in functions: open()*. Python 3.14 Documentation. <https://docs.python.org/3/library/functions.html#open>
- Python Software Foundation. (2026b). *pathlib — Object-oriented filesystem paths*. Python 3.14 Documentation. <https://docs.python.org/3/library/pathlib.html>
- Python Software Foundation. (2026c). *re — Regular expression operations*. Python 3.14 Documentation. <https://docs.python.org/3/library/re.html>
- Real Academia Española y Asociación de Academias de la Lengua Española. (2005/2023). *Diccionario panhispánico de dudas* (2.ª ed., versión provisional). <https://www.rae.es/dpd/>
- Real Academia Española y Asociación de Academias de la Lengua Española. (2009). *Nueva gramática de la lengua española*. Espasa.
- Zanettin, F. (2012). *Translation-driven corpora: Corpus resources for descriptive and applied translation studies*. St. Jerome.